

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Report

Laboratory work No. 5

The File System with Your Own Hands

Coșneanu Corina, st. gr. FAF-241

Checked by:
Reițman Patricia, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chișinău – 2026

Contents

1 LABORATORY OBJECTIVES AND THEORY **1**
1.1 Objective 1
1.2 Theoretical Background 1

2 PRACTICAL WORK AND OBSERVATIONS **2**
2.1 Part 1: The Directory Tree and File Types 2
2.1.1 Observe Answers (Part 1) 2
2.2 Part 2: Inodes, Links, and Disk Allocation 3
2.2.1 Observe Answers (Part 2) 4
2.3 Part 3: Permissions, Umask, and Special Bits 5
2.3.1 Observe Answers (Part 3) 5
2.4 Part 4: Disks and Kernel Pseudo-filesystems 7
2.4.1 Observe Answers (Part 4) 8

3 CONCLUSIONS **9**

1 LABORATORY OBJECTIVES AND THEORY

1.1 Objective

The main goal of this laboratory work is to gain hands-on experience with the Linux file system hierarchy, inode structures, hard and symbolic links, file and directory permissions (including special permission bits), disk partitioning, and pseudo-filesystems managed directly by the Linux kernel.

1.2 Theoretical Background

The Linux File System Hierarchy organizes all files and directories into a single tree rooted at `/`. Key concepts explored in this laboratory include:

- **Inodes (Index Nodes):** Data structures on disk that store file metadata (size, ownership, permissions, timestamp, block pointers), completely independent of the file name.
- **Hard Links vs. Symbolic Links:** A hard link is an additional directory entry pointing directly to an existing inode. A symbolic link (symlink) is a special file containing a text path reference to another file or directory.
- **Permissions & Umask:** File modes defined in octal or symbolic formats (`rwx`). The `umask` defines default permission masks when creating new files and directories.
- **Special Bits:** SetUID (`s`) executes a binary with the file owner's privileges, while the Sticky Bit (`t`) prevents unauthorized deletion in shared directories like `/tmp`.
- **Pseudo-filesystems:** Special filesystems such as `/proc` exposed by the kernel, providing real-time process and hardware state information.

2 PRACTICAL WORK AND OBSERVATIONS

2.1 Part 1: The Directory Tree and File Types

In this part, standard system paths were inspected to identify various file types based on the first character displayed by `ls -l`.

```
vboxuser@lab1:~/os-lab5$ ls /
bin   cdrom  etc   lib    lost+found  mnt  proc  run   snap  sys  usr
boot  dev    home  lib64  media       opt  root  sbin  srv   tmp  var
vboxuser@lab1:~/os-lab5$ ls -ld /etc /etc/passwd /bin /dev/null /tmp
lrwxrwxrwx   1 root root     7 Apr 20 08:46 /bin -> usr/bin
crw-rw-rw-   1 root root  1, 3 Oct 19 10:35 /dev/null
drwxr-xr-x 141 root root 12288 Oct 19 10:16 /etc
-rw-r--r--   1 root root  2778 Sep 28 14:59 /etc/passwd
drwxrwxrwt  15 root root   340 Oct 19 10:36 /tmp
vboxuser@lab1:~/os-lab5$ ls -l /dev | head
total 0
crw-r--r--  1 root     root     10, 235 Oct 19 10:35 autofs
drwxr-xr-x  2 root     root        500 Oct 19 10:35 block
drwxr-xr-x  2 root     root         80 Oct 19 10:34 bsg
crw-------  1 root     root     10, 234 Oct 19 10:34 btrfs-control
drwxr-xr-x  3 root     root         60 Oct 19 10:35 bus
lrwxrwxrwx+ 1 root     root          3 Oct 19 10:35 cdrom -> sr0
drwxr-xr-x  2 root     root       3740 Oct 19 10:35 char
crw-------  1 root     tty       5,   1 Oct 19 10:35 console
lrwxrwxrwx  1 root     root         11 Oct 19 10:34 core -> /proc/kcore
vboxuser@lab1:~/os-lab5$ file /usr/bin/ls /etc/passwd /bin
/usr/bin/ls: symbolic link to ../lib/cargo/bin/coreutils/ls
/etc/passwd: ASCII text
/bin:        symbolic link to usr/bin
vboxuser@lab1:~/os-lab5$ du -sh /var/log 2>/dev/null ; du -sh /usr 2>/dev/null
79M     /var/log
5.0G    /usr
```

Figure 2.1: Inspection of directory tree, file types, and disk usage

2.1.1 Observe Answers (Part 1)

1. **File Types from `ls -ld`:**
   - `/etc` → `d` (directory)
   - `/etc/passwd` → `-` (regular file)
   - `/bin` → `l` (symbolic link pointing to `usr/bin`)
   - `/dev/null` → `c` (character device)
   - `/tmp` → `d` (directory with sticky bit set: `drwxrwxrwt`)
2. **Target of `/bin`:** On modern Linux distributions, `/bin` is a symbolic link (`l`) pointing directly to `usr/bin`.
3. **Device Files in `/dev`:** Character device: `/dev/null` (or `/dev/tty`); Block device: `/dev/sda` (or `/dev/sda1`).
4. **Directory Size Comparison:** `/usr` is significantly larger (several GB, e.g. ∼3–8 GB) compared to `/var/log` (several MB, e.g. ∼20–100 MB), because `/usr` stores system binaries and libraries.

2.2 Part 2: Inodes, Links, and Disk Allocation

This section explores how hard links share inodes, how soft links store references, and how sparse files handle virtual size versus physical block usage.

```
vboxuser@lab1:~/os-lab5$ echo hello > notes.txt
vboxuser@lab1:~/os-lab5$ stat notes.txt
  File: notes.txt
  size: 6               Blocks: 8          IO Block: 4096   regular file
Device: 8,2     Inode: 290996      Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/vboxuser)   Gid: ( 1000/vboxuser)
Access: 2026-10-19 10:38:56.670247914 +0000
Modify: 2026-10-19 10:38:56.187244083 +0000
Change: 2026-10-19 10:38:56.187244083 +0000
 Birth: 2026-10-19 10:38:56.187244083 +0000
vboxuser@lab1:~/os-lab5$ ls -li notes.txt
290996 -rw-rw-r-- 1 vboxuser vboxuser 6 Oct 19 10:38 notes.txt
vboxuser@lab1:~/os-lab5$ ln notes.txt hard.txt
vboxuser@lab1:~/os-lab5$ ls -li notes.txt hard.txt
290996 -rw-rw-r-- 2 vboxuser vboxuser 6 Oct 19 10:38 hard.txt
290996 -rw-rw-r-- 2 vboxuser vboxuser 6 Oct 19 10:38 notes.txt
vboxuser@lab1:~/os-lab5$ rm notes.txt
vboxuser@lab1:~/os-lab5$ cat hard.txt ; ls -li hard.txt
hello
290996 -rw-rw-r-- 1 vboxuser vboxuser 6 Oct 19 10:38 hard.txt
vboxuser@lab1:~/os-lab5$
```

Figure 2.2: Creation and behavior of hard links

```
vboxuser@lab1:~/os-lab5$ ln -s hard.txt soft.txt
vboxuser@lab1:~/os-lab5$ ls -li hard.txt soft.txt
290996 -rw-rw-r-- 1 vboxuser vboxuser 6 Oct 19 10:38 hard.txt
291015 lrwxrwxrwx 1 vboxuser vboxuser 8 Oct 19 10:41 soft.txt -> hard.txt
vboxuser@lab1:~/os-lab5$ cat soft.txt
hello
vboxuser@lab1:~/os-lab5$ rm hard.txt
vboxuser@lab1:~/os-lab5$ cat soft.txt ; ls -l soft.txt
cat: soft.txt: No such file or directory
lrwxrwxrwx 1 vboxuser vboxuser 8 Oct 19 10:41 soft.txt -> hard.txt
```

Figure 2.3: Creation and behavior of symbolic (soft) links

```
vboxuser@lab1:~/os-lab5$ mkdir d ; ls -ld d
drwxrwxr-x 2 vboxuser vboxuser 4096 Oct 19 10:43 d
vboxuser@lab1:~/os-lab5$ mkdir d/x d/y ; ls -ld d
drwxrwxr-x 4 vboxuser vboxuser 4096 Oct 19 10:43 d
vboxuser@lab1:~/os-lab5$ echo x > one.txt ; ls -l one.txt ; du -h one.txt
-rw-rw-r-- 1 vboxuser vboxuser 2 Oct 19 10:43 one.txt
4.0K    one.txt
vboxuser@lab1:~/os-lab5$ truncate -s 1G big.img ; ls -lh big.img ; du -h big.img
-rw-rw-r-- 1 vboxuser vboxuser 1.0G Oct 19 10:43 big.img
0       big.img
vboxuser@lab1:~/os-lab5$ rm big.img
```

Figure 2.4: Directory link count origins and sparse file block allocation

2.2.1 Observe Answers (Part 2)

1. **Inode Identification in `stat`:** The number next to `Inode:` indicates the inode identifier, while `Links:` specifies how many filenames point to that inode.

2. **Hard Link Behavior after `rm`:** Removing `notes.txt` only removes the filename reference and decrements the inode link count. `hard.txt` still points to the same inode, maintaining a link count of 1; thus the underlying data remains intact.

3. **Symlink Failure after Target Removal:** `cat soft.txt` fails because `soft.txt` stores only the text path `"hard.txt"`. When `hard.txt` is removed, it becomes a dangling link. Its size is exactly 8 bytes (the length of the string `"hard.txt"`).

4. **Directory Link Count Origin:** A new directory `d` starts with 2 links (`d` in parent directory and `.` inside itself). Creating subdirectories `d/x` and `d/y` adds 2 more links because each subdirectory contains a `..` entry referencing `d` ($2 + 2 = 4$).

5. **Space vs Size:** Filesystems allocate disk space in minimum block units (typically 4 KB). A 2-byte file occupies one 4.0 KB block. Conversely, `truncate` creates a sparse file: virtual size is 1 GB, but physical block usage (`du`) is 0 KB until actual data blocks are written.

2.3 Part 3: Permissions, Umask, and Special Bits

Permissions determine read, write, and execute access for user, group, and others.

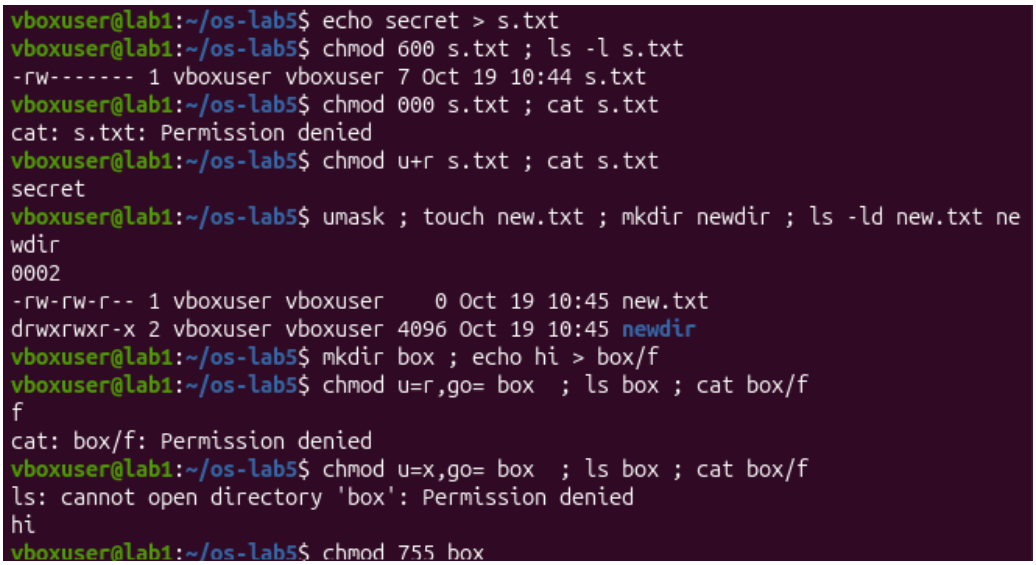


Figure 2.5: File permissions, umask calculation, and directory access rights

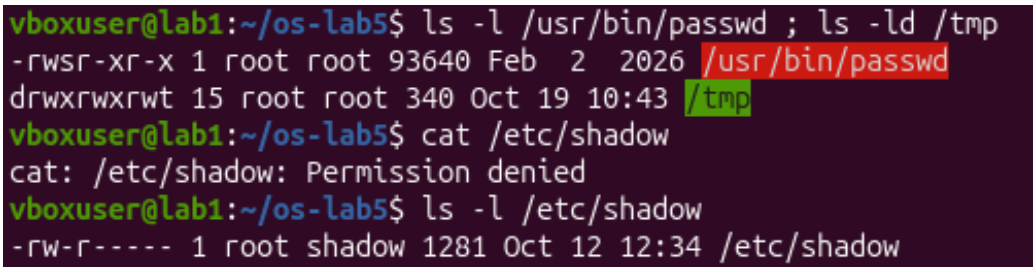


Figure 2.6: Special permission bits (SetUID on /usr/bin/passwd and Sticky Bit on /tmp)

2.3.1 Observe Answers (Part 3)

1. **Octal Notation & Real World Uses:**
   - `600` = `rw----` (Owner read/write; used for private SSH keys `/.ssh/id_rsa`).
   - `644` = `rw-r-r-` (Owner read/write, others read-only; used for public configuration/web files).
   - `755` = `rwxr-xr-x` (Owner full access, others read/execute; used for executable scripts and directories).
   - `700` = `rwx---` (Owner full access; used for private directories like `/.ssh`).
2. `chmod 000` **Ownership:** Ownership grants metadata modification rights (e.g. permission to run `chmod`), but does not bypass explicit permission bits (`--`). Since read access is disabled, `cat` is denied.
3. **Umask Impact:** A default `umask` of `0022` subtracts permissions from base creation masks (`666` for files, `777` for directories). `new.txt` becomes $666 - 022 = 644$ (`rw-r-r-`), while `newdir` becomes $777 - 022 = 755$ (`rwxr-xr-x`).

4. `box` **Experiment (`r` vs `x` on directories):**
    - Read (`r`): Allows listing file names inside the directory (`ls box`), but prevents reading contents or metadata without `x`.
    - Execute (`x`): Allows entering the directory and reading files inside it (`cat box/f`), but prevents listing directory contents.
5. **Special Bits (`s` and `t`):**
    - SetUID (`s`) on `/usr/bin/passwd`: Allows users to run the program with `root` privileges, which is required to write password hashes to `/etc/shadow`.
    - Sticky Bit (`t`) on `/tmp`: Allows any user to create temporary files, but prevents users from deleting files created by others.

2.4 Part 4: Disks and Kernel Pseudo-filesystems

Kernel data structures and hardware statistics are exposed as dynamic virtual files inside `/proc`.

```
vboxuser@lab1:~/os-lab5$ lsblk
NAME   MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
loop0    7:0    0  66.8M  1 loop /snap/core24/1643
loop1    7:1    0     4K  1 loop /snap/bare/5
loop2    7:2    0    20M  1 loop /snap/desktop-security-center/151
loop3    7:3    0  14.2M  1 loop /snap/desktop-security-center/206
loop4    7:4    0  66.8M  1 loop /snap/core24/2124
loop5    7:5    0 260.3M  1 loop /snap/firefox/8763
loop6    7:6    0  91.7M  1 loop /snap/gtk-common-themes/1535
loop7    7:7    0  18.8M  1 loop /snap/prompting-client/222
loop8    7:8    0  16.5M  1 loop /snap/firmware-updater/226
loop9    7:9    0   1.5M  1 loop /snap/hwctl/123
loop10   7:10   0   1.5M  1 loop /snap/hwctl/131
loop11   7:11   0  18.4M  1 loop /snap/prompting-client/228
loop12   7:12   0  11.8M  1 loop /snap/snap-store/1390
loop13   7:13   0  50.1M  1 loop /snap/snapd/27710
loop14   7:14   0 614.5M  1 loop /snap/gnome-46-2404/164
loop15   7:15   0   402M  1 loop /snap/mesa-2404/1839
loop16   7:16   0  44.7M  1 loop /snap/snapd/28254
loop17   7:17   0   828K  1 loop /snap/snapd-desktop-integration/391
sda      8:0    0    25G  0 disk
├─sda1   8:1    0     1G  0 part /boot/efi
└─sda2   8:2    0  23.9G  0 part /
sr0     11:0    1  1024M  1 rom
vboxuser@lab1:~/os-lab5$ df -h
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           519M  2.4M  517M   1% /run
/dev/sda2        24G  7.8G   15G  36% /
tmpfs           1.3G     0  1.3G   0% /dev/shm
efivarfs        256K   78K  174K  31% /sys/firmware/efi/efivars
tmpfs           1.3G  8.0K  1.3G   1% /tmp
none            1.0M     0  1.0M   0% /run/credentials/systemd-journald.service
none            1.0M     0  1.0M   0% /run/credentials/systemd-resolved.service
/dev/sda1       1.1G  6.4M  1.1G   1% /boot/efi
tmpfs           260M   80K  260M   1% /run/user/1000
vboxuser@lab1:~/os-lab5$ mount | grep " / "
/dev/sda2 on / type ext4 (rw,relatime)
```

Figure 2.7: Disk geometry, partition mounts, and filesystem types

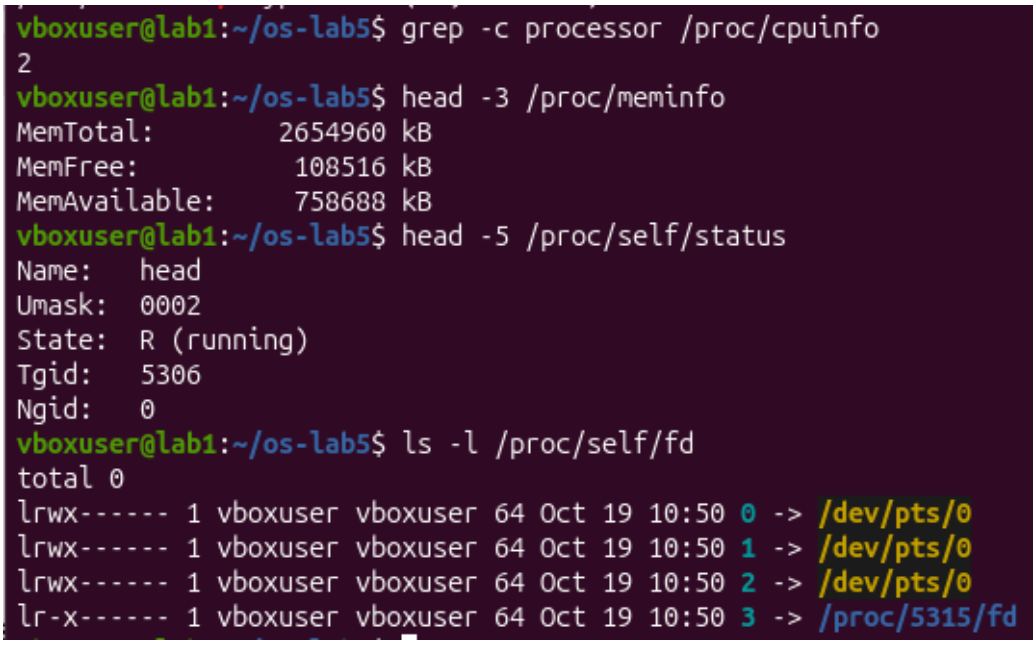


Figure 2.8: CPU, memory, process status, and file descriptors from /proc

2.4.1 Observe Answers (Part 4)

1. **Disk Architecture:**
   - Disk device name: `sda`.
   - Root partition: `sda2` mounted at `/`.
   - Filesystem type: `ext4`.
2. **Root Filesystem Usage:** The root filesystem usage is 36% occupied (7.8G used out of 24G).
3. **Hardware Info from `/proc`:**
   - CPU Cores: 2 cores.
   - Total RAM: 2654960 kB $\approx$ 2.65 GB.
4. **Program Name in `/proc/self/status`:** It displays `head`, which is the name of the process reading the file at that exact instance.

3 CONCLUSIONS

In this laboratory work, I gained practical familiarity with the low-level structure and management of the Linux file system. By investigating inodes, I observed how filenames act merely as pointers to underlying metadata structures, allowing multiple hard links to share the same physical storage blocks safely.

Furthermore, I explored the functional differences between file modes and directory permissions, analyzing how the read (`r`) and execute (`x`) bits control directory indexing versus file traversal. The practical inspection of SetUID (`s`) and Sticky Bit (`t`) demonstrated how Linux enforces security policies on executable binaries and shared directories like `/tmp`. Finally, examining pseudo-filesystems like `/proc` highlighted the UNIX philosophy of treating system state, process data, and hardware information as readable files.